MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name: Anton P George
Roll No: 13
Class: CSA
Course: PCCST501 – Computer Networks
Module: 1
Website chosen: Amazon (www.amazon.com)

When a user types www.amazon.com into a browser's address bar and presses Enter, a coordinated sequence of events unfolds across several layers of the Internet's protocol stack — from resolving the domain name to rendering the final page. This report explains that complete process step by step, addressing each part of the assignment in turn.

1. The TCP/IP Protocol Stack and the Role of Each Layer

Communication over the Internet is organised using the TCP/IP model, which splits the job of moving data into four layers. Each layer performs one specific task and hands its output to the layer below it (while sending) or above it (while receiving).

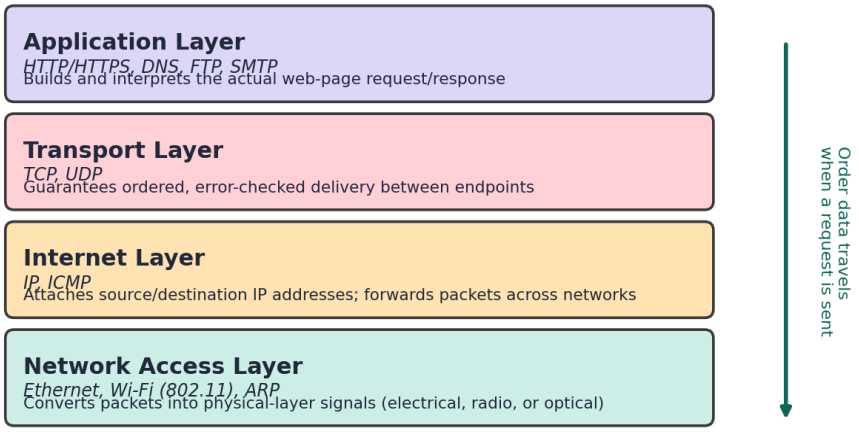


Figure 1: The four layers of the TCP/IP protocol stack

- **Application Layer –** Where the browser itself operates. It builds the HTTP/HTTPS request for the Amazon page and interprets the HTML/CSS/JavaScript that comes back. DNS resolution of www.amazon.com also takes place at this layer.
- **Transport Layer –** TCP takes the HTTP request, breaks it into numbered segments, and guarantees they reach Amazon's server complete and in order. It manages the three-way handshake that opens the connection.
- **Internet Layer –** IP wraps each segment into a packet and attaches the source and destination IP addresses. Routers use this layer to forward each packet across the many networks between the user and Amazon's servers.
- **Network Access Layer –** Converts IP packets into frames suited to the physical medium in use — Wi-Fi radio signals or Ethernet electrical/optical signals — and handles the actual transmission over the local network hop.

As the request travels down the stack, each layer wraps the data from the layer above it with its own header — a process called encapsulation. At Amazon's server, this is reversed layer by layer (decapsulation) until the original HTTP request is recovered.

2. Role of DNS in Converting the Domain Name into an IP Address

Browsers and routers do not understand domain names directly — all routing on the Internet is done using numeric IP addresses. The Domain Name System (DNS) acts as the Internet's directory service, translating a human-readable name such as www.amazon.com into the numeric address needed to reach the correct server.

1. The browser checks its own cache, then the operating system's cache, for a recent record of www.amazon.com.
2. If nothing is cached, a DNS resolver (usually run by the user's ISP or a public service such as 8.8.8.8) is queried.
3. The resolver follows the DNS hierarchy: a root server points it to the .com TLD servers, which point to Amazon's own authoritative name server.
4. The authoritative server returns the actual IP address. Amazon's site is served through a global CDN (Amazon CloudFront), so the address returned is typically that of the nearest edge server — for example, 205.251.242.103.
5. The address is cached locally (until its TTL expires) and handed back to the browser so the connection can proceed.

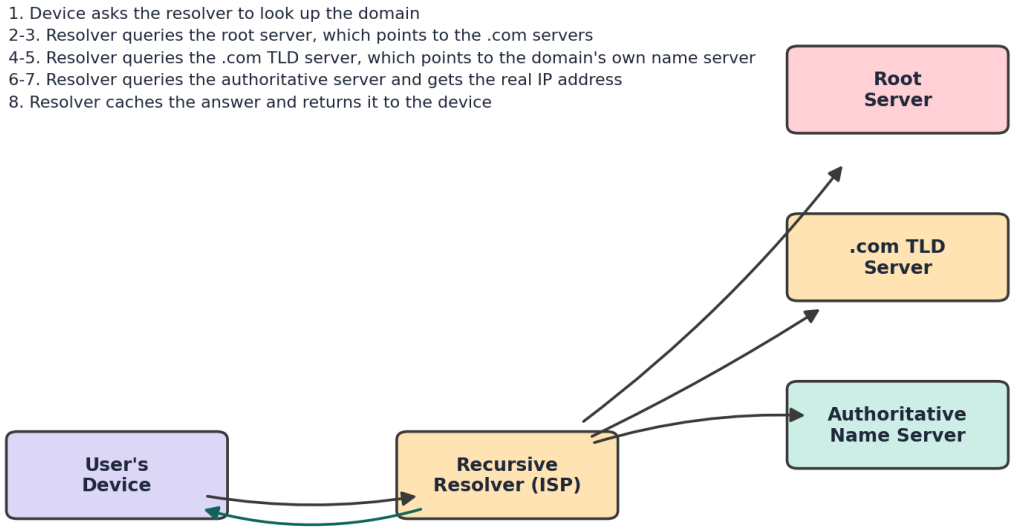


Figure 2: How a DNS lookup resolves www.amazon.com step by step

Without DNS, users would need to memorise numeric addresses for every website — which is exactly why DNS is often called the "phonebook of the Internet."

3. How HTTP is Used to Request and Receive the Webpage

Once the browser has Amazon's IP address and a TCP connection is open, it uses HTTP (in practice, its encrypted form HTTPS) to fetch the page. The browser sends a GET request for the desired page, e.g. GET /gp/product/example HTTP/1.1, along with headers such as Host and User-Agent. Amazon's web server processes the request and replies with a status line (such as 200 OK), response headers, and the HTML body of the page. The browser then parses the HTML and issues further GET requests for embedded resources — stylesheets, JavaScript, and product images — before finally rendering the complete page on screen.

Simplified request/response example

```
GET /gp/product/example HTTP/1.1
Host: www.amazon.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
```

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 61840
Set-Cookie: session-id=abc123; Secure; HttpOnly
<!DOCTYPE html>...(page content)...
```

In practice Amazon never uses plain HTTP — HTTPS wraps the same exchange inside a TLS encryption layer, so the browser and server first perform a TLS handshake to agree on encryption keys and verify Amazon's identity certificate. This prevents anyone on the network from reading or tampering with account details, payment information, or order history while the page loads.

4. How TCP Provides Reliable Communication for HTTP

HTTP relies entirely on TCP beneath it to make sure the request and response actually arrive intact. TCP achieves this through several mechanisms:

- **Three-way handshake –** SYN, SYN-ACK, ACK segments open the connection before any data is exchanged.
- **Sequence numbers –** every byte is numbered so segments can be reassembled correctly even if they arrive out of order.
- **Acknowledgements (ACKs) –** confirm receipt of data; a missing ACK signals a lost segment.
- **Retransmission –** a segment is resent automatically if its ACK does not arrive in time.
- **Flow control –** a sliding window prevents a fast sender from overwhelming a slower receiver.
- **Checksums –** detect corrupted data so it can be discarded and retransmitted.

5. Client–Server or Peer-to-Peer Architecture?

Amazon follows a Client–Server architecture. The user's browser (the client) always initiates the request, while Amazon operates centralised servers — backed by its own AWS data centres and the CloudFront CDN — that respond to requests, store product and account data, and serve pages. Clients never exchange data directly with one another; every request is routed through Amazon's infrastructure. This is the defining trait of a Client–Server system, unlike Peer-to-Peer (P2P) systems such as BitTorrent, where every node can act as both client and server, exchanging content directly with other peers with no central authority. Even though Amazon uses many distributed edge servers for performance, each server still only responds to client requests — it does not exchange data with other clients directly — so the architecture remains Client–Server, not P2P.

6. Comparison of HTTP and FTP

Aspect	HTTP	FTP
Purpose	Transfers web pages and resources for browsing/display	Transfers files between a client and server for storage/download
Connections used	A single connection carries both requests and data	Two separate connections: control (port 21) and data (port 20)
State	Stateless by default (state added via cookies)	Stateful – session persists for the duration of the connection
Authentication	Often none required for public content	Typically requires a username and password

Typical use case	Loading a webpage such as www.amazon.com	Uploading a website's files to a hosting server

7. How Cookies Improve the User Experience

HTTP is stateless — the server has no built-in memory of a user between separate requests. Cookies solve this by letting the server store a small piece of data on the user's browser, which is sent back automatically with every later request to that domain. This lets a site keep a user logged in and remember their preferences without asking them to re-enter information on every page.

Real-world example: When a shopper adds items to their Amazon cart, a cookie stores the cart's contents in the browser. This means the items stay in the cart even if the shopper closes the tab, browses other product pages, or returns the next day — without cookies, the cart would empty every time a new page loaded, since HTTP itself has no memory of the previous request.

8. Flow Diagram: Browser-to-Server Communication

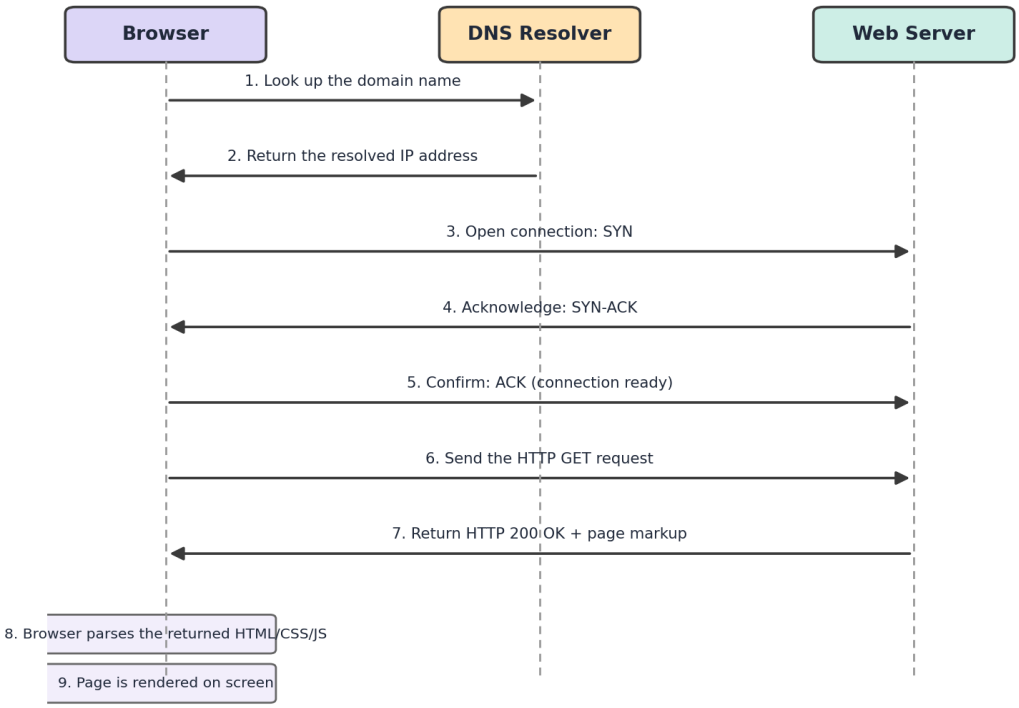


Figure 3: Complete communication flow from browser to web server and back for www.amazon.com

9. Conclusion

Loading a single page such as www.amazon.com depends on a tightly coordinated stack of application-layer protocols working invisibly in the background. DNS turns a memorable name into a routable address, TCP guarantees that the request and response survive the journey intact, IP finds the path across networks, and HTTP structures the actual exchange between browser and server. Cookies then add statefulness on top of an inherently stateless protocol, enabling persistent logins and features like a shopping cart. These application-layer protocols form the direct interface between everyday users and the Internet's infrastructure — every search, login, and purchase depends on them working correctly, quickly, and securely. Without this layered design, the modern web could not scale to billions of daily requests or remain reliable and secure for its users.